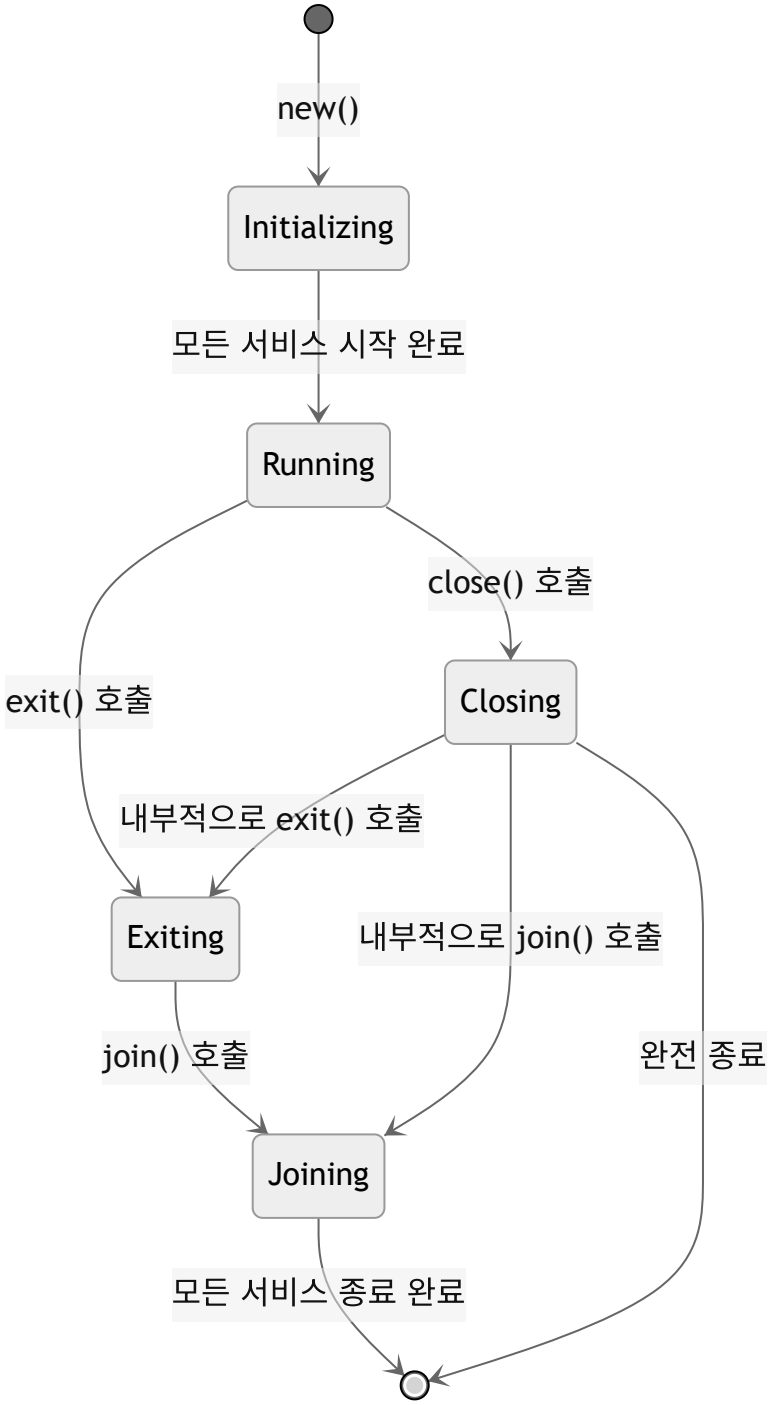
02-02 Validator 생명주기 및 종료 과정 분석

개요

Solana의 `Validator` 는 복잡한 분산 시스템으로서 여러 서비스와 스레드를 관리하며, 안전하고 순서가 있는 종료 과정이 필요합니다. 이 문서에서는 `Validator` 의 전체 생명주기와 `exit()` , `join()` , `close()` 함수를 통한 종료 과정을 상세히 분석합니다.

Validator 생명주기 개요

생명주기 단계



각 단계별 특징

1. **Initializing (초기화)**: `Validator::new()` 실행 중
2. **Running (실행)**: 모든 서비스가 활성화되어 정상 동작
3. **Exiting (종료 신호)**: 종료 신호 전파 및 새로운 작업 중단
4. **Joining (대기)**: 모든 서비스 스레드 종료 대기
5. **Terminated (완료)**: 모든 리소스 정리 완료

Exit 메커니즘 분석

Exit 구조체

```
#[derive(Default)]
pub struct Exit {
exited: bool,
exits: Vec<Box<dyn FnOnce() + Send + Sync>>,
}

impl Exit {
pub fn register_exit(&mut self, exit: Box<dyn FnOnce() + Send + Sync>) {
if self.exited {
exit();
} else {
self.exits.push(exit);
}
}

pub fn exit(&mut self) {
self.exited = true;
for exit in self.exits.drain(..) {
exit();
```

```
}
}
}
```

Exit 메커니즘의 특징

1. **콜백 기반**: 각 서비스가 종료 콜백을 등록
2. **즉시 실행**: 이미 종료된 상태에서 등록하면 즉시 실행
3. **일괄 실행**: `exit()` 호출시 모든 등록된 콜백을 순차 실행

Validator::exit() 함수 분석

함수 구현

```
// Used for notifying many nodes in parallel to exit
pub fn exit(&mut self) {
self.validator_exit.write().unwrap().exit();

// drop all signals in blockstore
self.blockstore.drop_signal();
}
```

동작 과정

1. **종료 신호 전파**

```
self.validator_exit.write().unwrap().exit();
```

- 모든 등록된 서비스에 종료 신호 전송
- 각 서비스의 종료 콜백 함수 실행

2. **블록스토어 신호 정리**

```
self.blockstore.drop_signal();
```

- 블록스토어의 모든 신호 채널 정리
- 새로운 블록 처리 중단

종료 신호 등록 예시

초기화 과정에서 각 서비스가 종료 콜백을 등록하는 방식:

```
// PubSub 서비스 종료 콜백 등록
let (trigger, pubsub_service) = PubSubService::new(
config.pubsub_config.clone(),
&rpc_subscriptions,
rpc_pubsub_addr,
);
config
.validator_exit
.write()
.unwrap()
```

```
.register_exit(Box::new(move || trigger.cancel()));


// 전역 exit 신호와 연결
let exit = exit.clone();
config
.validator_exit
.write()
.unwrap()
.register_exit(Box::new(move || exit.store(true, Ordering::Relaxed)));
```

Validator::join() 함수 분석

함수 구현 및 종료 순서

```
pub fn join(self) {
// 1. 핵심 데이터 구조 해제
drop(self.bank_forks);
drop(self.cluster_info);


// 2. PoH 서비스 종료
self.poh_service.join().expect("poh_service");
drop(self.poh_recorder);
```

```
// 3. RPC 서비스 종료
if  let Some(json_rpc_service) = self.json_rpc_service {
json_rpc_service.join().expect("rpc_service");
}


if  let Some(pubsub_service) = self.pubsub_service {
pubsub_service.join().expect("pubsub_service");
}


self.rpc_completed_slots_service
.join()
.expect("rpc_completed_slots_service");


// 4. 트래킹 서비스 종료
if  let Some(optimistically_confirmed_bank_tracker) =
self.optimistically_confirmed_bank_tracker
{
optimistically_confirmed_bank_tracker
.join()
.expect("optimistically_confirmed_bank_tracker");
}


// 5. 트랜잭션 관련 서비스 종료
```

```
if  let Some(transaction_status_service) = self.transaction_status_service {
transaction_status_service
.join()
.expect("transaction_status_service");
}

if  let Some(rewards_recorder_service) = self.rewards_recorder_service {
rewards_recorder_service
.join()
.expect("rewards_recorder_service");
}

if  let Some(cache_block_meta_service) = self.cache_block_meta_service {
cache_block_meta_service
.join()
.expect("cache_block_meta_service");
}

// 6. 모니터링 서비스 종료
if  let Some(system_monitor_service) = self.system_monitor_service {
system_monitor_service
.join()
.expect("system_monitor_service");
```

```
}

if  let Some(sample_performance_service) = self.sample_performance_service {
sample_performance_service
.join()
.expect("sample_performance_service");
}

if  let Some(entry_notifier_service) = self.entry_notifier_service {
entry_notifier_service
.join()
.expect("entry_notifier_service");
}

// 7. 스냅샷 서비스 종료
if  let Some(s) = self.snapshot_packager_service {
s.join().expect("snapshot_packager_service");
}

// 8. 네트워크 서비스 종료
self.gossip_service.join().expect("gossip_service");
// 9. QUIC 엔드포인트 정리
if  let Some(repair_quic_endpoint) = &self.repair_quic_endpoint {
```

```
repair::quic_endpoint::close_quic_endpoint(repair_quic_endpoint);

}

self.serve_repair_service

.join()

.expect("serve_repair_service");

if  let Some(repair_quic_endpoint_join_handle) = self.repair_quic_endpoint_join_

self.repair_quic_endpoint_runtime

.map(|runtime| runtime.block_on(repair_quic_endpoint_join_handle))

.transpose()

.unwrap();

};



// 10. 통계 및 메트릭 서비스 종료

self.stats_reporter_service

.join()

.expect("stats_reporter_service");

self.blockstore_metric_report_service

.join()

.expect("ledger_metric_report_service");



// 11. 계정 관련 서비스 종료

self.accounts_background_service

.join()
```

```
.expect("accounts_background_service");
self.accounts_hash_verifier
.join()
.expect("accounts_hash_verifier");


// 12. Turbine QUIC 엔드포인트 정리
if  let Some(turbine_quic_endpoint) = &self.turbine_quic_endpoint {
solana_turbine::quic_endpoint::close_quic_endpoint(turbine_quic_endpoint);
}


// 13. 핵심 처리 유닛 종료
self.tpu.join().expect("tpu");
self.tvu.join().expect("tvu");
if  let Some(turbine_quic_endpoint_join_handle) = self.turbine_quic_endpoint_joi
self.turbine_quic_endpoint_runtime
.map(|runtime| runtime.block_on(turbine_quic_endpoint_join_handle))
.transpose()
.unwrap();
}


// 14. 데이터 완성 서비스 종료
self.completed_data_sets_service
.join()
```

```
.expect("completed_data_sets_service");


// 15. IP 에코 서버 종료
if  let Some(ip_echo_server) = self.ip_echo_server {
ip_echo_server.shutdown_background();
}


// 16. Geyser 플러그인 서비스 종료
if  let Some(geyser_plugin_service) = self.geyser_plugin_service {
geyser_plugin_service.join().expect("geyser_plugin_service");
}


// 17. PoH 타이밍 보고 서비스 종료
self.poh_timing_report_service
.join()
.expect("poh_timing_report_service");
}
```

서비스 종료 순서 분석

종료 순서의 중요성

Validator의 서비스 종료 순서는 의존성과 데이터 무결성을 고려하여 설계되었습니다:

1단계: 핵심 데이터 구조 해제

```
drop(self.bank_forks);

drop(self.cluster_info);
```

- **목적**: 새로운 트랜잭션 처리 중단
- **효과**: 다른 서비스들이 더 이상 새로운 작업을 시작하지 않음

2단계: PoH 서비스 종료

```
self.poh_service.join().expect("poh_service");

drop(self.poh_recorder);
```

- **목적**: 시간 증명 생성 중단
- **중요성**: PoH는 모든 트랜잭션 처리의 기반이므로 먼저 중단

3단계: 외부 인터페이스 종료

- RPC 서비스들 (JSON-RPC, PubSub)
- 외부 클라이언트와의 연결 차단

4-6단계: 내부 서비스 종료

- 트랜잭션 상태 추적
- 보상 기록
- 시스템 모니터링

7-9단계: 네트워크 서비스 종료

- 가십 프로토콜
- 복구 서비스

- QUIC 엔드포인트

10-12단계: 백그라운드 서비스 종료

- 통계 수집
- 계정 처리
- 메트릭 보고

13단계: 핵심 처리 유닛 종료

```
self.tpu.join().expect("tpu");

self.tvu.join().expect("tvu");
```

- **중요성**: 가장 마지막에 종료하여 모든 진행 중인 작업 완료 보장

종료 순서 다이어그램

Validator::close() 함수 분석

함수 구현

```
pub fn close(mut self) {
self.exit();
self.join();
}
```

동작 특징

1. **편의 함수**: `exit()` 와 `join()` 을 순차적으로 호출
2. **완전 종료**: 종료 신호 전송부터 모든 스레드 대기까지 한 번에 처리
3. **소유권 이동**: `self` 를 소비하여 재사용 방지

리소스 정리 과정

메모리 관리

1. **Arc<RwLock<>> 해제**

```
drop(self.bank_forks);
drop(self.cluster_info);
drop(self.poh_recorder);
```

- 공유 데이터 구조의 참조 카운트 감소
- 마지막 참조가 해제되면 자동으로 메모리 정리

2. **스레드 종료 대기**

```
service.join().expect("service_name");
```

- 각 서비스 스레드의 완전한 종료 대기
- 예외 발생시 패닉으로 처리하여 불완전한 종료 방지

네트워크 리소스 정리

1. **QUIC 엔드포인트 정리**

```
if let Some(repair_quic_endpoint) = &self.repair_quic_endpoint {
repair::quic_endpoint::close_quic_endpoint(repair_quic_endpoint);
}
```

2. **소켓 정리**

```
if let Some(ip_echo_server) = self.ip_echo_server {
ip_echo_server.shutdown_background();
}
```

비동기 런타임 정리

```
if let Some(repair_quic_endpoint_join_handle) = self.repair_quic_endpoint_join_
self.repair_quic_endpoint_runtime
.map(|runtime| runtime.block_on(repair_quic_endpoint_join_handle))
.transpose()
.unwrap();
```

```
};
```

- Tokio 런타임에서 실행 중인 비동기 작업들의 완료 대기
- 런타임 자체의 정리

오류 처리 및 복구

종료 과정에서의 오류 처리

1. **expect() 사용**

```
self.poh_service.join().expect("poh_service");
```

- 서비스 종료 실패시 즉시 패닉
- 불완전한 종료 상태 방지

2. **unwrap() 사용**

```
.transpose()
.unwrap();
```

- 비동기 작업 완료 실패시 패닉
- 시스템 무결성 보장

강제 종료 시나리오

1. **시그널 처리**: SIGTERM, SIGINT 등의 시스템 시그널
2. **타임아웃**: 일정 시간 내 정상 종료되지 않을 경우

3. **치명적 오류**: 복구 불가능한 오류 발생시

성능 고려사항

종료 시간 최적화

1. **병렬 종료**: 독립적인 서비스들의 동시 종료
2. **의존성 순서**: 의존성이 있는 서비스들의 순차 종료
3. **타임아웃 설정**: 각 서비스별 적절한 종료 타임아웃

메모리 사용량

1. **점진적 해제**: 서비스별로 순차적인 메모리 해제
2. **참조 카운트**: Arc를 통한 안전한 메모리 관리
3. **즉시 해제**: drop()을 통한 명시적 메모리 해제

모니터링 및 디버깅

로깅

각 서비스 종료시 적절한 로깅을 통해 종료 과정 추적:

```
info!("Shutting down validator services...");

// 각 서비스 종료 후

info!("Service {} shut down successfully", service_name);
```

메트릭

종료 과정의 성능 메트릭 수집:

- 각 서비스별 종료 시간
- 전체 종료 소요 시간
- 메모리 해제 패턴

결론

Solana Validator의 생명주기 관리는 복잡한 분산 시스템의 안전한 종료를 위한 정교한 설계를 보여줍니다.

핵심 특징

1. **순서가 있는 종료**: 의존성을 고려한 체계적인 서비스 종료 순서
2. **완전성 보장**: 모든 서비스의 완전한 종료 대기
3. **리소스 정리**: 메모리, 네트워크, 파일 등 모든 리소스의 적절한 정리
4. **오류 처리**: 종료 과정에서의 오류를 패닉으로 처리하여 불완전한 상태 방지

이러한 설계는 Solana가 고성능을 유지하면서도 안정적인 운영을 가능하게 하는 핵심 요소입니다.